TP2 : MILops

1) Explications du fonctionnement de la pipeline

Les agenst deamons:
ils écoutent une queue Clear ML et exécutent les tâches qui y sont mises en file d'attente. D'abord, l'agent 1 (NB_Agent1) qui exécuter les étapes de la pipeline (step_prepare_data, step_train, step_register_model), car elles déclarent execution_queue="default

Puis,l'agent 2 (NB Agent2) qui lance les tâches de contrôle : le watcher NB_B et le run du contrôleur de pipeline

La pipeline (NB_C_pipeline)

Les cellules 1 et 2 permettent d'importer clearML et les cle api

La Cellule 3 lance le bootstrap du dataset CIFAR-10 vers ClearML.

```
# Extraction locale via torchvision
print("Extraction locale...")
train_ds = datasets.CIFAR10(root=data_root, train=True, download=True)
test_ds = datasets.CIFAR10(root=data_root, train=False, download=True)

print(f"Train: {len(train_ds)} | Test: {len(test_ds)}")

# Envoi vers ClearML
dataset_dir = os.path.join(data_root, "cifar-10-batches-py")
new_dataset = Dataset.create(
    dataset_project="Image-Classifier/Gradio-Retrain-Demo-IABD",
    dataset_name="cifar10-base"
)
new_dataset.add_files(path=dataset_dir)
new_dataset.upload()
new_dataset.finalize()

print("CIFAR-10 importé avec succès sur ClearML !")
```

la Cellule 4 : le contrôleur

```
task = Task.init(project_name=PROJECT, task_name="retrain_pipeline_controller", task_type=Task.TaskTypes.controller)
print("Controller Task ID:", task.id)
```

- Task.init crée la tâche contrôleur : c'est elle qui va orchestrer le DAG, pas entraîner.
- set_packages ne s'applique qu'à l'environnement du contrôleur ; il ne propage pas torch/torchvision aux tâches composants (il faudrait PipelineDecorator.set_default_packages). Et les versions torch==2.14.1 / torchvision==0.29.1 n'existent pas.

Les composants

```
@PipelineDecorator.component(
    name="step_register_model",
    execution_queue="default",
    return_values=["improved","new_model_id"]
)
```

Chaque @PipelineDecorator.component(...) transforme une fonction Python en tâche Clear ML indépendante :

- execution_queue="default" : le code part sur un agent qui écoute default (NB_Agent1).
- return_values=[...] : ClearML sérialise et transmet les valeurs de retour à l'étape suivante.
- Le code de la fonction est rejoué à distance : les imports doivent donc être refaits dans la fonction.

step_prepare_data()
ds = Dataset.get(dataset_name="image-feedback", dataset_project=PROJECT)
return ds.get_local_copy(force_download=True)

- Récupère le dataset de feedback dans le cache local de l'agent.
- force_download=True contourne un cache corrompu (mais coûte du temps).
- En cas d'absence → retourne "", et l'entraînement se fera sur CIFAR seul.
- Problème : aucune version n'est épinglée ; c'est « la dernière au moment de l'exécution », pas celle qui a déclenché la pipeline.

step_train(feedback_dir)

- DEVICE = "cpu" forcé (3 epochs, batch 128, lr 1e-3).
- Transform train (RandomCrop + flip) / test (normalisation seule).
- ResNet18 weights=None, fc remplacée par 10 sorties → entraînement from scratch, pas de transfer learning.
- Récupère cifar10-base depuis ClearML, et recrée le sous-dossier cifar-10-batches-py si les fichiers sont à plat.
- Charge CIFAR10(train=True/False) avec download=False.
- Si feedback_dir existe : ImageFolder + ConcatDataset avec CIFAR. C'est ici que le dossier unlabeled de NB-A casse l'entraînement (11e classe).
- Boucle d'entraînement : log valid/acc à chaque epoch, garde le best.
- Sauvegarde le state_dict dans ~/clearml_outputs/resnet18_<ts>.pt.
- t.update_output_model(...) → upload l'artefact et le déclare comme modèle de sortie avec les tags candidate, resnet18.
- Retourne (model_id, {"valid_acc": best}).

step_register_model(model_id, metrics_dict)

- Cherche les modèles production du projet (only_published=True).
- Lit valid_acc dans leurs métadonnées (0.0 si absent).
- improved = new_acc >= prod_acc.
- m.publish() puis écrit valid_acc en métadonnée.
- Si improved : archive l'ancien production, tague le nouveau production. Sinon : le tague staging.
- Retourne (improved, model_id).

C'est le model registry / promotion gate, avec les défauts déjà cités (comparaison >=, archive non atomique, métadonnée manquante).
Le DAG

```python
@PipelineDecorator.pipeline(
    name="retrain_pipeline",
    project=PROJECT,
    version="1.0",
    pipeline_execution_queue=None
)
def pipeline():
    fb = step_prepare_data()
    mid, metrics = step_train(fb)
    step_register_model(mid, metrics)
```

4. Cellule 5 : lancement

```python
# Déclenchement (non bloquant côté notebook, le contrôleur part sur l'agent)
pipeline()
```

Le pipeline déclenche le contrôleur. IDeux modes d'exécution coexistent donc :

- Manuel : pipeline() dans le notebook → contrôleur local.
- Automatique : NB_B clone la tâche retrain_pipeline_controller et le met en fil d'attente sur controller → c'est NB_Agent2 qui doit la reprendre. Or les composants, eux, exigent la queue default (NB_Agent1).

NB-B — Trigger (SDK) — ClearML Community

Imports et configuration

```python
# NB-B — Watcher "latest by date" (no status filter)
# Déclenche la pipeline dès qu'un NOUVEL ID de dataset (par date) apparaît,
# sans filtrer par statut (draft/final).


import time, traceback
from pathlib import Path
from clearml import Task, Dataset

# -------- CONFIG --------
PROJECT = "Image-Classifier/Gradio-Retrain-Demo-IABD"       # Projet du DATASET (comme affiché dans l'UI)
DATASET_NAME = "image-feedback"                          # Nom EXACT du dataset
PIPELINE_TASK_ID = "33714fe795ea49a8adffdfb55770e1b1"  # Run "template" clonable de la pipeline
QUEUE = "controller"                                        # Queue ClearML pour exécuter la pipeline
POLL_SECS = 15
# Use a writable local directory instead of /content, which is only valid in Colab.
STATE_FILE = Path.home() / ".clearml" / ".last_dataset_id"     # Mémoire dernier ID vu
STATE_FILE.parent.mkdir(parents=True, exist_ok=True)
# ------------------------
```

Initialisation de la tâche

ctrl = Task.init(project_name=PROJECT, task_name="dataset_watcher_latest", task_type=Task.TaskTypes.controller)

Détecteur de changement

```python
while True:
    try:
        matches = _list_matching(DATASET_NAME, PROJECT)

        # Affichage debug
        print("\n🔎 Matching datasets:")
        if not matches:
            print("  ❌ None")
        else:
            for d in sorted(matches, key=lambda x: x.get("created","")):
                print(f"  - id={d['id']} | created={d.get('created')} | status={d.get('status')}")

        latest = _latest_by_date(matches)
        if latest is None:
            print("[poll] No versions yet.")
        else:
            ds_id = latest["id"]
            print(f"[poll] Latest by date: id={ds_id} | created={latest.get('created')} | status={latest.get('status')}")
            if ds_id and ds_id != last_seen:
                print(f"🎯 NEW LATEST DATASET D
                run_id = _clone_enqueue_pipeline(PIPELINE_TASK_ID, QUEUE, run_name_suffix=f"__ds_{ds_id[:8]}")
                print(f"✅ Enqueued pipeline run: {run_id} on queue '{QUEUE}'")
                last_seen = ds_id
                _write_last_id(last_seen)
            else:
                print("[poll] Nothing new.")
```

(constant) PIPELINE_TASK_ID: Literal['33714fe795ea49a8adffdfb55770e1b1'] = "33714fe795ea49a8adffdfb55770e1b1"

C'est la boucle de surveillance du watcher , elle tourne indéfiniment dans le kernel du notebook, interroge ClearML toutes les 15 s, et déclenche la pipeline quand elle voit un nouvel ID de dataset.

NB-A — Inference + Feedback (Gradio) — ClearML Community

Configuration et chemins

```
# -------------------- CONFIG --------------------
PROJECT = "Image-Classifier/Gradio-Retrain-Demo-IABD"
DATASET_NAME = "image-feedback"
# Use a writable local directory instead of /content, which is only valid in Colab.
BASE_DIR = Path.home() / "clearml_feedback"
FEEDBACK_DIR = BASE_DIR / "feedback"
STATE_FILE = BASE_DIR / "state.json"    # to count stored samples
BASE_DIR.mkdir(parents=True, exist_ok=True)
FEEDBACK_DIR.mkdir(parents=True, exist_ok=True)
```

Architecture du modèle

```
# -------------------- MODEL LOADING --------------------
def build_resnet18(num_classes=10):
    m = models.resnet18(weights=None)
    m.fc = nn.Linear(m.fc.in_features, num_classes)
    return m

CLASSES = [
    "airplane","automobile","bird","cat","deer",
    "dog","frog","horse","ship","truck"
]
```

Prétraitement et prédiction

Le prétraitement doit être identique à celui de l'entraînement, sinon les prédictions sont faussées.

```
TFM = transforms.Compose([
    transforms.Resize((32,32)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]),
])

@torch.no_grad()
def predict_image(pil: Image.Image) -> Tuple[str, float]:
    pil = pil.convert("RGB")
    x = TFM(pil).unsqueeze(0).to(DEVICE)  # [1,3,32,32]
    logits = model(x)
    prob = torch.softmax(logits, dim=1)[0]
    conf, idx = torch.max(prob, dim=0)
    return CLASSES[int(idx)], float(conf.item())
```

Création d'une version de Dataset tous les 5

```python
    st = _load_state()
    st["stored"] += 1
    did = False
    dsid = st.get("last_version_id")

    if st["stored"] >= 5:
        try:
            parent_id = None
            parent_ds = None
            # Try to get parent (latest 'image-feedback' in this project)
            try:
                parent_ds = Dataset.get(dataset_name=DATASET_NAME, dataset_project=PROJECT)
                parent_id = parent_ds.id
            except Exception:
                parent_ds, parent_id = None, None

            # If parent exists but might be draft, try finalize (best-effort)
            if parent_ds is not None:
                try:
                    parent_ds.finalize()
                except Exception:
                    pass  # ok if already final or cannot finalize

            # Try create with parent; if it fails (draft parent), create without parent
            try:
                new = Dataset.create(
                    dataset_name=DATASET_NAME,
                    dataset_project=PROJECT,
                    parent_datasets=[parent_id] if parent_id else None,
                )
```

C'est le mécanisme de versioning par lots : on attend 5 retours utilisateur pour éviter une version par image.

```python
with gr.Blocks(title="ResNet18 CIFAR-10 – Predict & (optional) Store") as demo:
    gr.Markdown("### Upload an image (CIFAR-10-like), then **Predict**. Optionally **Store** with a true label to collect feedback.")
    with gr.Row():
        in_img = gr.Image(type="pil", label="Input image")
        with gr.Column():
            do_store   = gr.Checkbox(label="Store feedback", value=False)
            true_label = gr.Dropdown(choices=CLASSES + ["unlabeled"], value="unlabeled", label="True label (if storing)")
            btn        = gr.Button("Predict (and Store if checked)")
    with gr.Row():
        out_label = gr.Textbox(label="Predicted label")
        out_conf  = gr.Number(label="Confidence", precision=4)
    out_status = gr.Textbox(label="Status")

    btn.click(predict_and_store, inputs=[in_img, do_store, true_label], outputs=[out_label, out_conf, out_status])

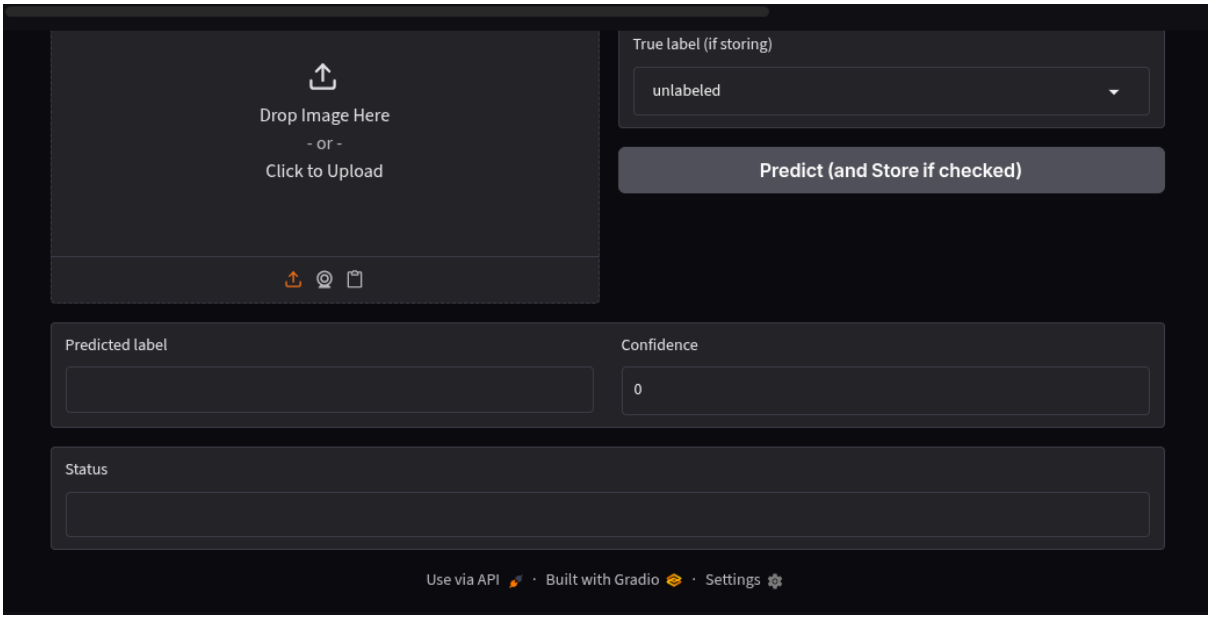

demo.launch(share=False)  # set share=True if you need a public link
```

Ce code créer un UI minimal.

En somme, au démarrage, le script construit l'architecture ResNet18 (adaptée à 10 classes) puis va chercher dans ClearML un modèle publié à charger. L'idée est que l'app serve toujours la dernière version issue de la pipeline.

NB-A est le point d'entrée du système : une app Gradio qui

- charge le dernier modèle publié depuis ClearML,
- prédit la classe CIFAR-10 d'une image,
- enregistre éventuellement le feedback de l'utilisateur (label corrigé),
- tous les 5 feedbacks, crée une nouvelle version du Dataset image-feedback — ce qui doit déclencher la pipeline de réentraînement.



2) 3 problèmes de cette pipeline

Problème 1 : Déclenchement du réentraînement non fiable (NB-B)

- Constat : NB-B utilise un polling manuel avec Dataset.list_datasets() pour comparer l'ID du dernier dataset à celui stocké dans ~/.clearml/.last_dataset_id. Il ne s'agit pas d'un vrai TriggerScheduler comme annoncé dans le markdown.
- Causes :
- Aucun tag auto-feedback n'est posé par NB-A sur les datasets créés.
- Le dataset_id du lot déclencheur n'est pas passé à la pipeline.
- Si le fichier d'état est perdu (last_seen = ""), le dernier dataset existant est considéré comme « nouveau » et déclenche un réentraînement inutile.
- Conséquence : Des runs parasites, non liés au vrai lot de feedback, ainsi qu'un déclenchement non reproductible.

Problème 2 : Feedback invalide et duplication des données (NB-A / NB-C)

- a) Un label invalide qui fait planter l'entraînement
- Dans NB-A, il était possible d'enregistrer une image avec le label unlabeled. Celle-ci était sauvegardée dans feedback/unlabeled/.
- Dans NB-C, step_train utilise torchvision.datasets.ImageFolder qui déduit les classes à partir des noms de sous-dossiers. unlabeled/ devenait donc une 11e classe avec l'index 10.
- Le ResNet18 n'ayant que 10 sorties, torch.nn.CrossEntropyLoss levait alors l'erreur : IndexError: Target 10 is out of bounds for dimension 1 with size 10, faisant directement planter le réentraînement.
- b) Ré-upload intégral du dossier à chaque version
- Avant correction, NB-A faisait new.add_files(str(FEEDBACK_DIR)) sans jamais déplacer les fichiers. Tout le contenu du dossier était donc ré-envoyé à chaque nouvelle version.

- Concrètement : la v1 contenait 5 images, la v2 contenait les 5 anciennes + 5 nouvelles (10), la v3 contenait 15, etc. Le dataset grossissait linéairement avec des doublons à l'intérieur de chaque version.
- Correction apportée : Refus du label unlabeled dans l'UI et le callback, stockage dans pending/ (uniquement les nouveaux échantillons), add_files uniquement sur pending/, puis déplacement vers uploaded/<dataset_id>/ après la création de la version. On vérifie désormais via Added: 5 dans ClearML que seul le lot courant est uploadé.

Problème 3 : Incohérence entre inférence et promotion du modèle (NB-A / NB-C)
- Mauvais tag utilisé pour l'inférence : step_register_model promeut le meilleur modèle en lui attribuant le tag production. Or, NB-A interroge Model.query_models avec le tag candidate pour charger le modèle à servir. L'application pouvait donc servir un modèle rejeté au lieu du modèle réellement promu.
- Promotion trop laxiste : Le critère utilisé est improved = new_acc >= prod_acc. Avec >=, un modèle ayant exactement la même accuracy que le modèle en production le remplace sans aucun gain. De plus, s'il n'existe aucun modèle production, prod_acc vaut 0.0, ce qui fait que n'importe quel modèle entraîné est automatiquement promu.
- Fallback dangereux : Si aucun modèle tagué candidate n'est trouvé, NB-A bascule silencieusement sur un ResNet18 initialisé aléatoirement. L'interface Gradio continue de fonctionner et d'afficher des prédictions avec une fausse confiance, sans avertir l'utilisateur que ce modèle n'a jamais été entraîné.
- Conséquence : Risque de servir un modèle dégradé, non promu, voire totalement aléatoire en production.

Problème : Feedback invalide et duplication des données
Ce problème touche principalement le NB-A et le NB-C.
Constat
- Le dossier unlabeled casse l'entraînement : Dans NB_A, si l'utilisateur ne remplit pas le label, l'image va directement dans feedback/unlabeled/. NB_C utilise datasets.ImageFolder, qui va donc créer une 11e classe alors que ResNet18 n'a que 10 sorties. torch.nn.CrossEntropyLoss lèvera alors une erreur Target 10 is out of bounds et fera planter le step_train.

NB-A - store_feedback

```
label = true_label.strip() if (true_label and str(true_label).strip()) else "unlabeled"
dest_dir = FEEDBACK_DIR / label
```

- Ré-upload intégral à chaque version :
À chaque fois que le compteur atteint 5, NB_A fait new.add_files(str(FEEDBACK_DIR)). Le dossier contient déjà toutes les images des versions précédentes. Résultat : à la version 2, on re-uploade les 5 premières + les 5 nouvelles, etc. Le dataset grossit exponentiellement et on duplique les mêmes fichiers à chaque cycle.

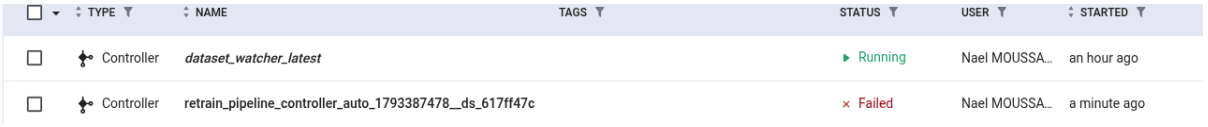
NB-A - version_dataset_if_needed

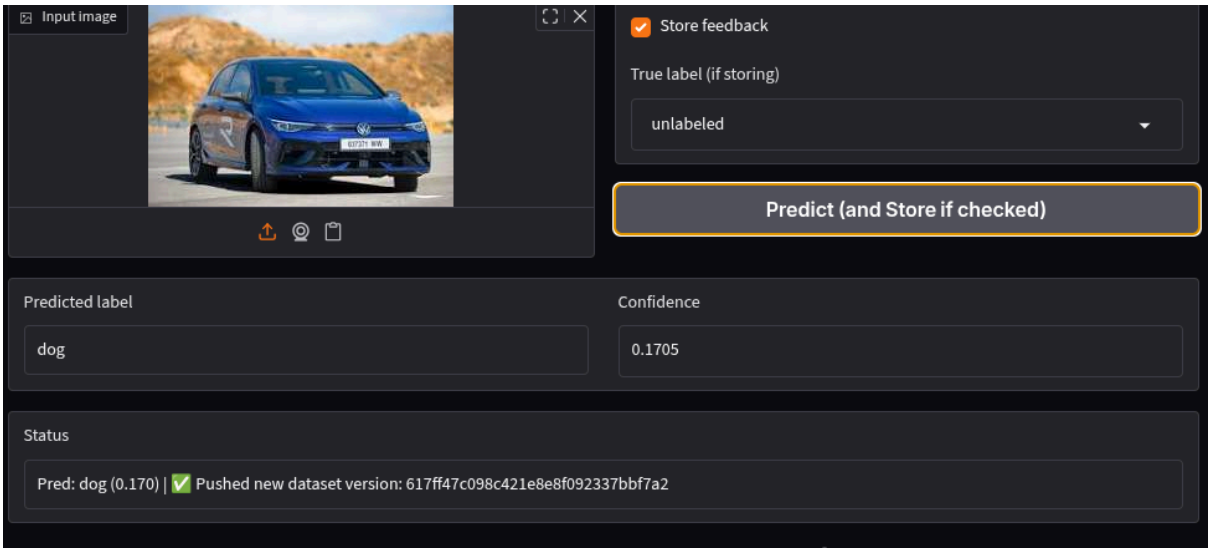
```
new.add_files(str(FEEDBACK_DIR))
try:
    new.upload()
except Exception as e:
    print("upload warning:", e)

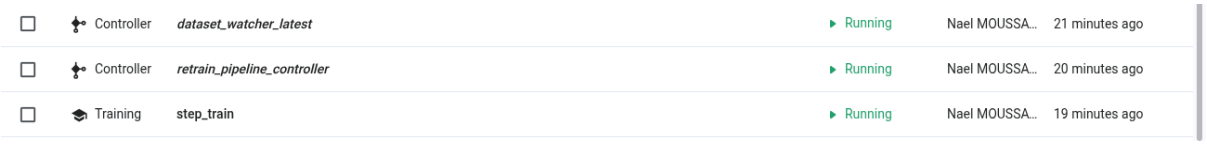
new.finalize()
```

Conséquence : Dataset qui explose en taille, temps d'entraînement inutiles, et risque très élevé de planter la pipeline dès qu'un utilisateur laisse le label à unlabeled.

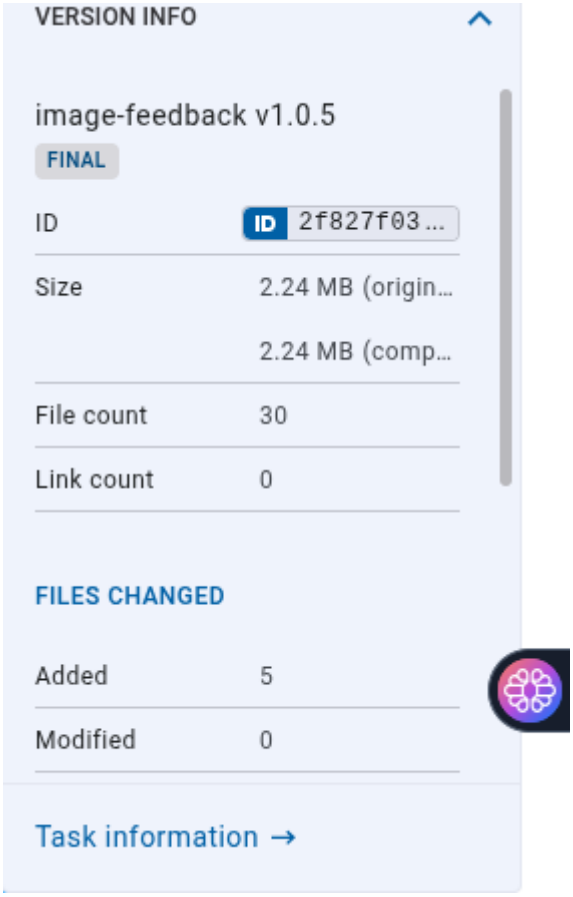
Nous avons alors regle ce probleme comme vous pouvez le voir :







Dans l'interface ClearML, le plus important à regarder pour ce cas précis, ce n'est pas File count, c'est FILES CHANGED > Added.
Le fait que File count monte est normal et souhaitable. Ce qu'on voulait éviter, ce n'était pas que le dataset grandisse au fil des versions (il doit bien contenir tout l'historique du feedback), c'était qu'à chaque version on ré-uploade l'intégralité.
Or ici, seul Added: 5 compte pour l'upload. C'est propre. Chaque nouvelle version n'ajoute bien que son lot de 5, sans duplication.



VERSION INFO

image-feedback v1.0.4

FINAL

ID	ID 617ff47c ...
Size	1.94 MB (origin...
	1.94 MB (comp...
File count	25
Link count	0

FILES CHANGED

Added	5
Modified	0

Task information →

VERSION INFO

image-feedback v1.0.3

FINAL

ID	ID b5a77542 ...
Size	1.63 MB (origin...
	1.64 MB (comp...
File count	20
Link count	0

FILES CHANGED

Added	5
Modified	0

Task information →